

Object Oriented Programming with Java 1 (2026)

Assignment 3

Niels-Christian Borbjerg
nielsb@setur.fo

1 Problem description

In this assignment you are to create and implement an application for drawing colored geometric shapes in the console. The shapes we are dealing with are circles and rectangles. We define a point to be a tuple of natural numbers (x, y) meaning $x, y \in \mathbb{N}$. We define a circle by its center point $p = (x, y)$ and its radius $r \in \mathbb{N}$ — for simplicity we only use radii of natural numbers. A rectangle is defined by its upper left and lower right corners $p_1 = (x_1, y_1)$ and $p_2 = (x_2, y_2)$ respectively, again with $x_1, y_1, x_2, y_2 \in \mathbb{N}$. A color is often represented as a triple (red, green, blue) where each entry is a color channel, and each entry is an integer between 0 and 255:

$$c = (r, g, b)$$

with $r, g, b \in [0, 255]$. For example if $c = (255, 0, 0)$ then c represents the color red, if $c = (0, 0, 0)$ then c represents black, while if $c = (255, 255, 255)$ then c represents white.

2 Classes and implementation

Your implementation can (should) use the following details as its point of departure. Make sure to read the entire document before coding as there might be a mutual and non-linear dependence between sections.

2.1 A grid based drawing canvas

In order for your program to actually draw shapes, you need to be able to place shapes on a canvas. One way to interpret such a canvas is to view it as a Cartesian coordinate system — what kind of data structure is suited for this representation? – this grid should be an internal representation of your drawing canvas encapsulated in a Grid class. The Grid class should extend the template seen in listing 1.

```

1 public class Grid {
2     // Fields
3     private int h;
4     private int w;
5     ... // Keep your internal representation of the grid somewhere here
6
7     // Constructor parameterized by height and width of the grid
8     public Grid(int h, int w) {...}
9     public void drawRectangle(Rectangle rectangle) {...}
10    public void drawCircle(Circle circle) {...}
11    public void renderGrid() {...}
12 }

```

Listing 1: Some fields and methods associated with the Grid class.

The tasks of the methods shown in listing 1 are the following

- Given a Rectangle object the drawRectangle method is responsible for drawing it on the canvas.
- Given a Circle object the drawCircle method is responsible for drawing it on the canvas.
- The renderGrid method is responsible for actually displaying the shapes drawn on the canvas.

Note that when dealing with pictures and images on a computer, the classic Cartesian plane interpretation is often times flipped in such a way that the origin is found in the upper left corner, while only keeping a single quadrant as seen in figure 1.

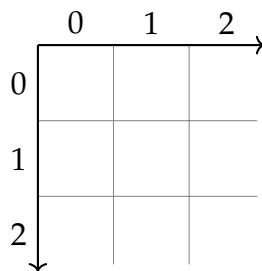


Figure 1: Illustration of modified Cartesian grid.

Each entry of such a grid can also be viewed as a so-called 'pixel', that when given a color or intensity forms an image.

When displaying the grid using the renderGrid method it might be a good idea to clear the console first, in order to have a clean canvas for drawing, this can be achieved by the method shown in listing 2.

```

1 private void ClearConsole() {
2     try {
3         if (System.getProperty("os.name").contains("Windows")) {
4             new ProcessBuilder("cmd", "/c", "cls").inheritIO().start().waitFor();
5         } else {
6             Runtime.getRuntime().exec("clear");
7         }
8     } catch (Exception ex) {}
9 }

```

Listing 2: Method for clearing the console.

2.2 Grid cells

As we see in figure 1 each entry of the grid can be viewed as a cell which can be interpreted as an object in its own right, thus you are to implement a `GridCell` class. This class should store information regarding what is currently inhabiting the specific cell, this could for example be information regarding color and content. Furthermore the `GridCell` class should expose a method for printing its content to screen called `printGridCell` satisfying the signature

`printGridCell : () -> ()`

See the documentation of the method shown in listing 4 for an example of how to print some content with a specific coloring.

2.3 Geometry classes

Furthermore we need classes representing our geometric objects, these include a `Point` class, and, as seen in listing 1, a `Rectangle` and `Circle` class for representing shapes. In order to implement these it is a good idea think of the definitions mentioned in the problem description which are the necessary abstractions for grasping the nature of their geometry. Both the `Rectangle` and `Circle` should have a `isInside` method that given a query point returns true or false depending on whether or not the query point is inside the shape, i.e. the method should follow the following signature

`isInside : Point -> Boolean`

Additionally both classes should be given a color. This for example means the `Rectangle` class could use the code stub shown in listing 3 as a reference point.

```

1 public class Rectangle {
2     ...
3     public Rectangle(Point p1, Point p2, Color color) {...}
4     public boolean isInside(Point p) {...}
5 }

```

Listing 3: The starting point of the `Rectangle` class.

```

1 public class Color {
2     ...
3     /**
4      * Returns an ANSI escape sequence that can be used to color text
5      * that is printed to the console. The following snippet will - if
6      * BG represents a blue background color and FG represents a red
7      * foreground color - print a red X on a blue background. The
8      * ANSI_RESET variable is used to reset any coloring options.
9      * {@snippet :
10     * String ANSI_RESET = "\u001B[0m";
11     * System.out.print(Color.colorCode(bg, fg) + "X" + ANSI_RESET);
12     * }
13     *
14     * @param    bg The background color
15     * @param    fg The foreground color
16     * @return    A String representing the desired fore-
17     *            and background color
18     */
19     public static String colorCode(Color bg, Color fg) {
20         /*
21          * ANSI escape sequence for background and foreground
22          * color is captured by the code:
23          * \033[48;2;<br>;<bg>;<bb>;38;2;<fr>;<fg>;<fb>m
24          */
25         return String.format(
26             "\033[48;2;%d;%d;%d;38;2;%d;%d;%dm",
27             bg.r, bg.g, bg.b, fg.r, fg.g, fg.b);
28     }
29 }

```

Listing 4: Method for generating a color code for the foreground and background of text printed to the console. It assumes that the color class has the public fields R, G and B containing the respective color channels.

2.4 A Color class

In order to represent colors you should implement a Color class. This class should have fields for storing a red, green and blue color channel. Furthermore the Color class should have a static method for mixing two different colors, meaning when given two different colors a new mixed color should be returned, i.e. satisfy the signature

```
MixColors : Color -> Color -> Color
```

Lastly the Color class should have another static method for generating a color code that can be used to color the shapes when displayed, e.g. it can be used in the printGridCell method of the GridCell class. This color code generator is shown in listing 4.

2.5 Example

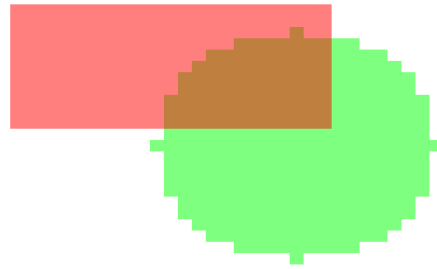
An example of a main method and the result of running it is seen in figure 2. Try to recreate something similar yourself. In the example shown the default color of the grid is white, and the content of each grid cell is a string consisting of three

```

1 public class Assignment3 {
2     public static void main(String[] args) {
3         Grid grid = new Grid(40, 40);
4         Rectangle rectangle = new Rectangle(
5             new Point(13, 5),
6             new Point(23, 27),
7             new Color(255, 0, 0));
8         Circle circle = new Circle(
9             new Point(25, 25),
10            10,
11            new Color(0, 255, 0));
12        grid.drawCircle(circle);
13        grid.drawRectangle(rectangle);
14        grid.renderGrid();
15    }
16 }

```

(a)



(b)

Figure 2: An example of the program and its output. The main method seen in (a) produced the image in (b) in the console.

spaces. When the content just consists of spaces, it is only the background color that is interesting.

3 Submission

The assignment should be handed in on your GitHub repository in a folder named Assignment3. Be sure to commit and push your solution.

The Moodle Assignment 3 submission is a text file named Assignment3.txt which only contains the URL for your GitHub repository.

The assignment should be handed in no later than 23:59 on May 1st.